FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Mark Anthony G. Petrimetre
Section:	AN-41
Professor:	Crisola Tan
Date:	10/2/26

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

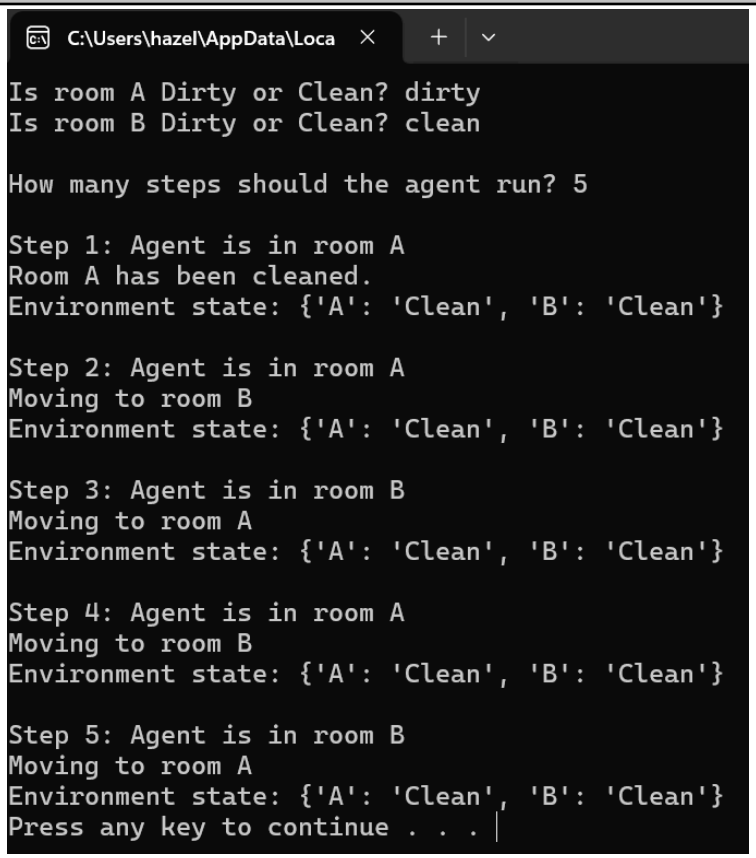
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

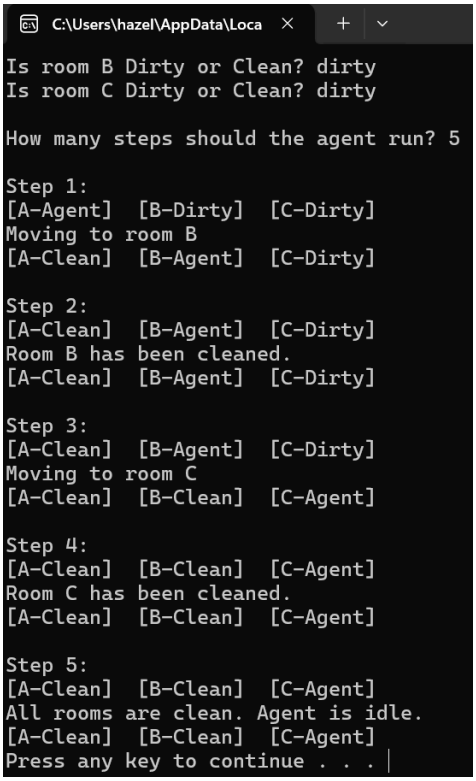
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Code:

```
class VacuumAgent:
    def __init__(self, start_room="A"):
```

```
        self.current_room = start_room

    def perceive_and_act(self, environment):
        """
        Rule 1: If current room is Dirty -> clean it.
        Rule 2: If current room is Clean -> switch to the other room.
        """
        current_state = environment[self.current_room]

        if current_state == "Dirty":
            # Action: Clean
            environment[self.current_room] = "Clean"
            action = f"Cleaned room {self.current_room}"
        else:
            # Action: Move
            action = self.move()

        return action

    def move(self):
        old_room = self.current_room
        self.current_room = "B" if self.current_room == "A" else "A"
        return f"Moved from Room {old_room} to Room {self.current_room}"



def is_dirty(room, environment):
    return environment.get(room) == "Dirty"

def clean_room(room, environment):
    environment[room] = "Clean"


def main():
    print("=== Rule-Based Agent Simulation ===")
```

```
    state_a = input("Enter state for Room A (Clean/Dirty):
").strip().capitalize()
    state_b = input("Enter state for Room B (Clean/Dirty):
").strip().capitalize()

    environment = {
        "A": "Dirty" if state_a == "Dirty" else "Clean",
        "B": "Dirty" if state_b == "Dirty" else "Clean"
    }

    start_pos = input("Enter initial agent location (A/B):
").strip().upper()
    if start_pos not in ["A", "B"]:
        start_pos = "A"

    steps = int(input("Enter number of simulation steps: "))

    agent = VacuumAgent(start_room=start_pos)

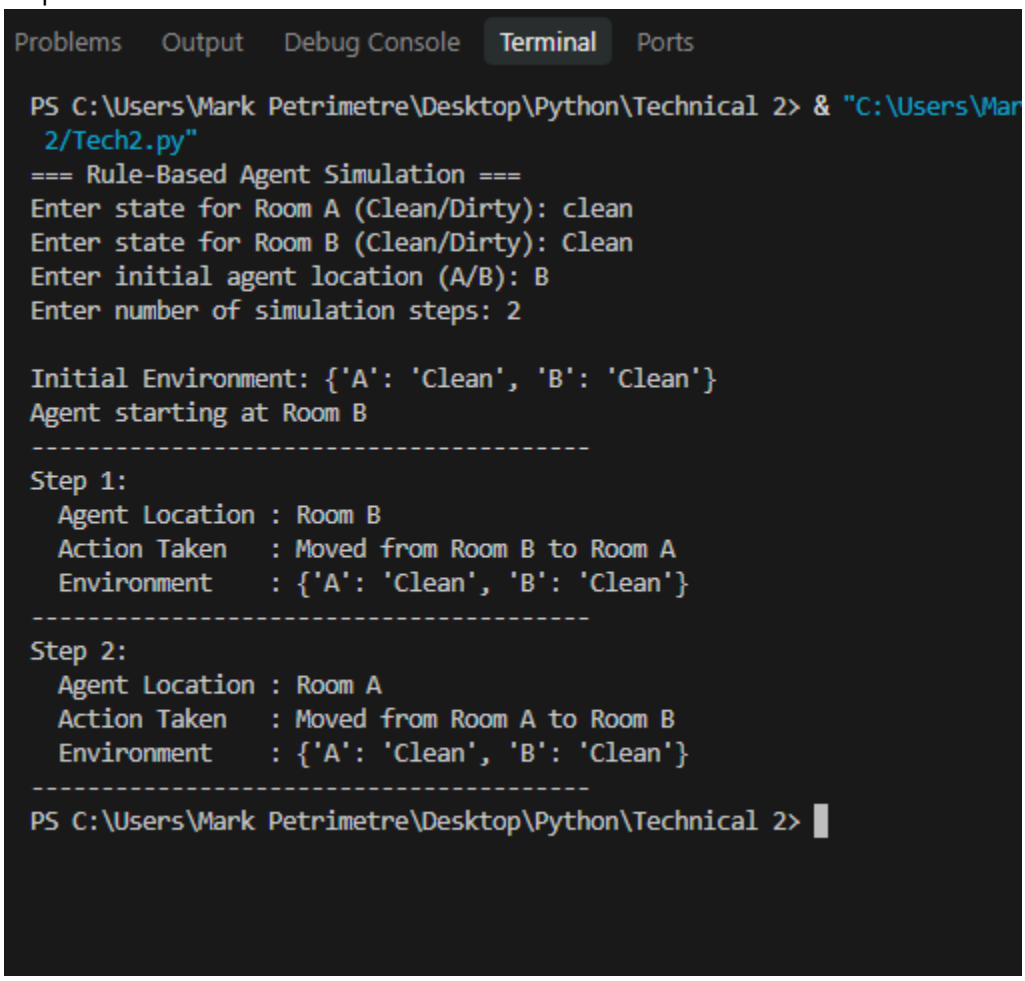
    print("\nInitial Environment:", environment)
    print(f"Agent starting at Room {agent.current_room}\n" + "-" * 40)

    for step in range(1, steps + 1):
        loc_before = agent.current_room
        action_taken = agent.perceive_and_act(environment)

        print(f"Step {step}:")
        print(f"  Agent Location : Room {loc_before}")
        print(f"  Action Taken   : {action_taken}")
        print(f"  Environment    : {environment}")
        print("-" * 40)


if __name__ == "__main__":
    main()
```

Output:



Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
    - Rule 1 (Condition-Action): If status(current_room) == "Dirty", execute clean().
    - Rule 2: If status(current_room) == "Clean", execute move() to navigate to the adjacent room.
2. How does the agent decide when to clean and when to move?
    - The decision cycle relies on immediate percepts via perceive_and_act(). It queries the current state of its immediate location inside the environment dictionary. If the lookup

returns "Dirty", priority is given to cleaning; otherwise, it branches to the movement subroutine to search the other partition.

3. What happens if both rooms are already clean at the start?
    - The agent enters an oscillation loop. Because condition 1 never triggers, Rule 2 executes repeatedly across each step, causing the agent to ping-pong back and forth between Room A and Room B without altering any states.
4. How would you modify the agent to handle **more than two rooms**?
    - Replace hardcoded binary transitions with an ordered sequence or topological map (e.g., a list of rooms ['A', 'B', 'C', ...] or an adjacency graph).
    - For systematic coverage, increment an index circularly using modulo arithmetic ((curr_index + 1) % total_rooms) or track visited/dirty room queues to avoid redundant traversals.

Answer the Questions ⇩